Experiment 3

Title: Real-time Undo/Redo System for Text Editor using Stack Data Structure

Objectives:

To design and implement an undo/redo mechanism for a text editing application.

To utilize the Stack data structure effectively to manage document changes for undo and redo operations.

To ensure real-time responsiveness for undo, redo, and display document state operations.

To provide a clear and intuitive user experience for managing document revisions.

Problem Statement:

Text editing applications require a robust and efficient undo/redo functionality to allow users to revert or reapply changes. The challenge lies in designing a system that can track document modifications, store them for potential reversal, and reapply them accurately, all while maintaining a responsive user interface. This system must handle multiple changes and their corresponding undo/redo operations in a sequential manner.

Outcomes:

Upon successful implementation, the system will:

Provide a functional undo operation that reverts the most recent change.

Provide a functional redo operation that reapplies the most recently undone change.

Accurately display the current state of the document after any undo or redo action.

Demonstrate the effective application of the Stack data structure in managing sequential operations.

Software and Hardware Requirements:

Software:

Operating System: Windows 10/11, macOS, or Linux distribution.

Programming Language: Python (or Java, C++, etc., depending on implementation choice).

IDE/Text Editor: VS Code, PyCharm, IntelliJ IDEA, or similar.

Hardware:

Processor: Intel Core i3 or equivalent.

RAM: 4GB or more.

Storage: 100MB free disk space.

Date of Completion: October 6, 2025

Assessment grade/marks and assessor's sign: (To be filled by assessor)

Theory Concept in brief:

The core of this system relies on the Stack data structure, a Last-In, First-Out (LIFO) collection. Two stacks are primarily used: an "undo stack" and a "redo stack." When a change is made, it is pushed onto the undo stack. When an undo operation occurs, the change is popped from the undo stack and pushed onto the redo stack. Conversely, a redo

operation pops a change from the redo stack and pushes it back onto the undo stack. This mechanism ensures that changes are tracked and reversible in the correct order.

Algorithm:

Initialize: Create two empty stacks: undo_stack and redo_stack. Initialize current_document_state.

Make a Change (e.g., apply_change(new_state)):

a. Push current_document_state onto undo_stack.
b. Update current_document_state to new_state.
c. Clear redo_stack. Undo Action (undo()).

a. If undo_stack is not empty:
i. Push current_document_state onto redo_stack.
ii. Pop previous_state from undo_stack.
iii. Update current_document_state to previous_state.
b. Else: Display "No more changes to undo." Redo Action (redo()).

a. If redo_stack is not empty:
i. Push current_document_state onto undo_stack.
ii. Pop next_state from redo_stack.
iii. Update current_document_state to next_state.
b. Else: Display "No more changes to redo." Display Document State (display_document()).

a. Print current_document_state.

Flowchart:

Program:-

```
class TextEditor:
    def __init__(self):
        self.current_document_state = ""
        self.undo_stack = []
        self.redo_stack = []

    def make_change(self, new_content):
        self.undo_stack.append(self.current_document_state)
        self.current_document_state = new_content
        self.redo_stack = []  # Clear redo stack on new change
        self.display_document_state()

    def undo_action(self):
        if self.undo_stack:
            self.redo_stack.append(self.current_document_state)
            self.current_document_state = self.undo_stack.pop()
            self.display_document_state()
        else:
            print("Nothing to undo.")

    def redo_action(self):
        if self.redo_stack:
            self.undo_stack.append(self.current_document_state)
            self.current_document_state = self.redo_stack.pop()
            self.display_document_state()
        else:
            print("Nothing to redo.")

    def display_document_state(self):
        print(f"Current Document: '{self.current_document_state}'")

# Usage Example:
editor = TextEditor()
editor.make_change("Hello")
editor.make_change("Hello World")
editor.undo_action()
editor.make_change("Hello Python")
editor.undo_action()
editor.redo_action()
```